



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2022/2023

Sensores e Atuadores
Relatório: Trabalho Lab 05

Turma: 11D

Grupo04:

Nome: Rodrigo Nicolau

Número: 50728

Nome: Robert Sbircea

Número: 50732

Nome: Fábio Pestana

Número: 50756

Data: 8 de Dezembro 2022

Lab-05

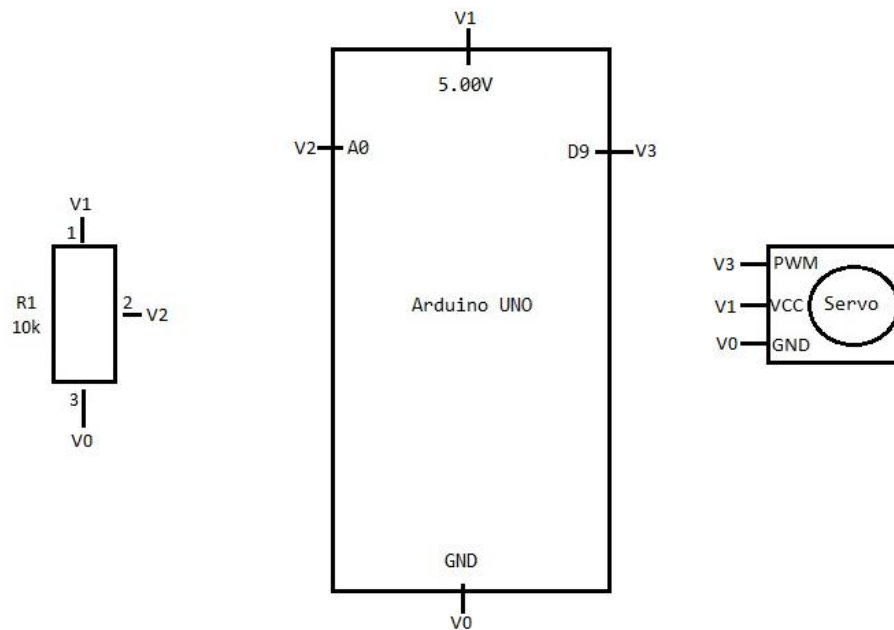
Objetivos: Introduzir aos alunos conceitos práticos sobre a programação de novos atuadores e sensores (servo, sonar, buzzer, piezo e LED RGB) através da placa Arduino, em C⁺⁺.

Materiais:

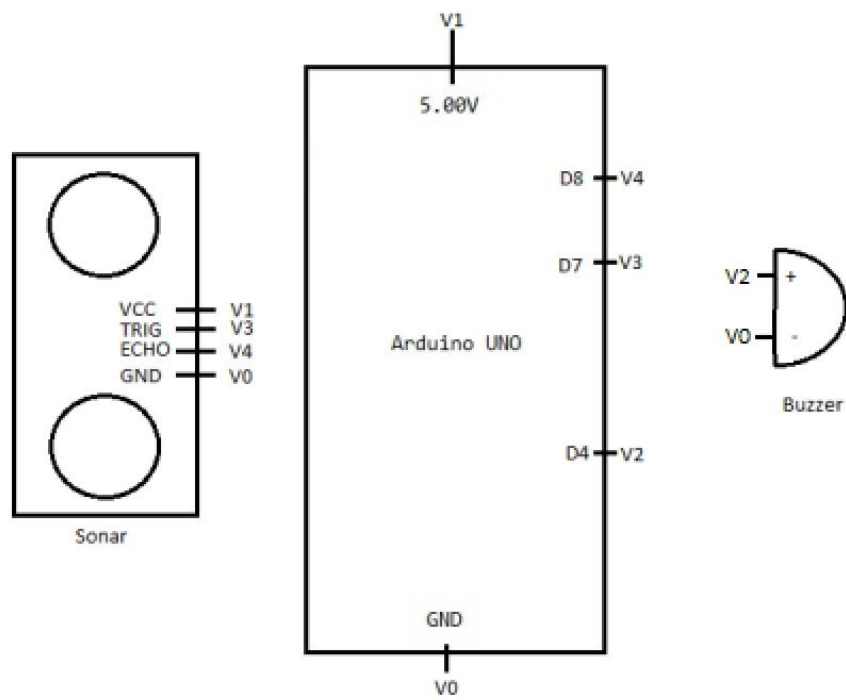
- *Breadboard*
- Arduino Uno
- Resistências
- LDR
- *LED RGB*
- Servo
- Sonar
- Buzzer
- Piezo
- Osciloscópio
- Cabos
- PC

Tarefas:

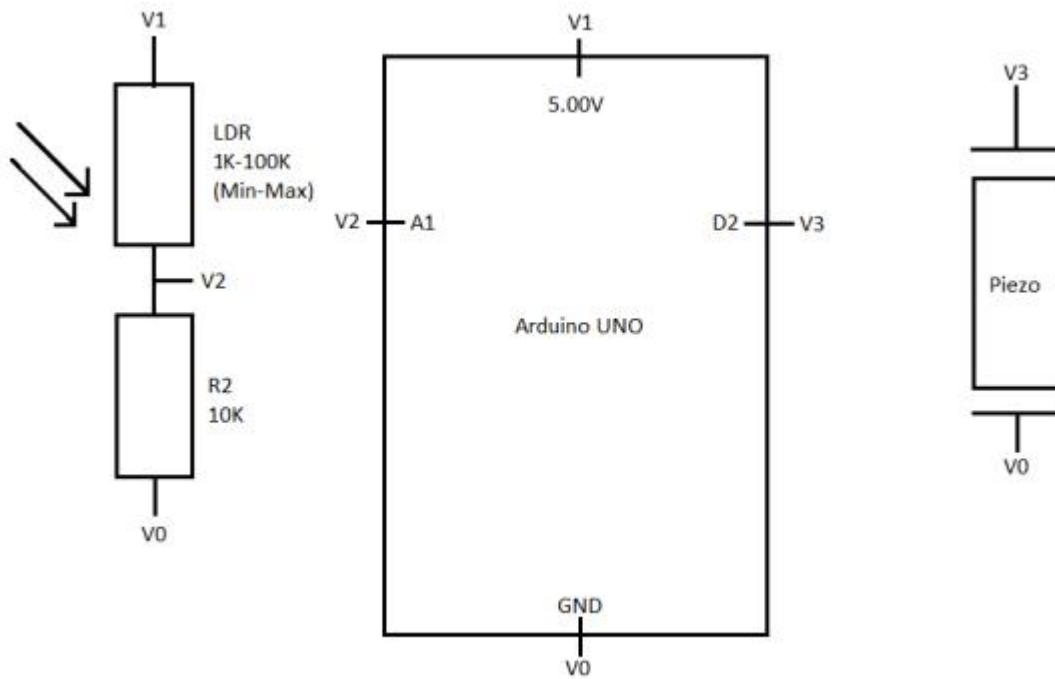
Circuito 1:



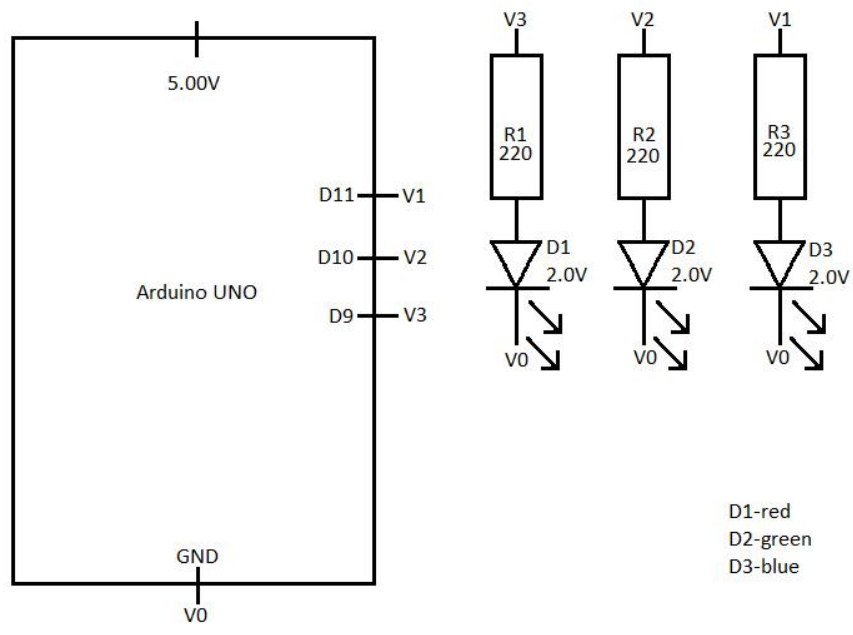
Circuito 2:



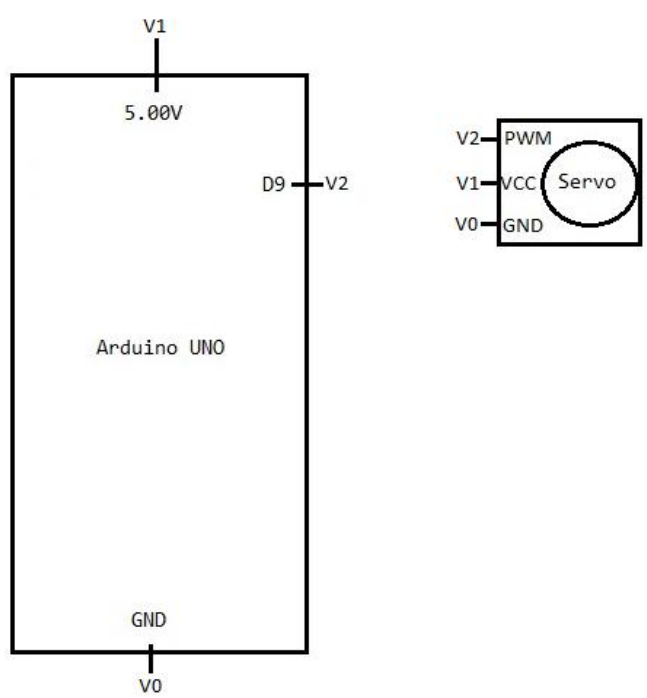
Circuito 3:



Circuito 4:



Circuito 5:



Circuito 1:

```
#include <Servo.h>

#define potpin A0
#define servo 9

Servo nameServo;
int pot=0;
int x,val;

void setup()
{
  Serial.begin(9600);
  nameServo.attach(9);
}

void loop()
{
  x = analogRead(potpin);
  val= map(x, 0,1023,-90, 90); //val= map(x, 0,1023,0, 180);
  nameServo.write(val);
  delay(15);
  Serial.println(x/1023.0);
  Serial.println(val);
}
```

O código em cima apresentado tem como objetivo, fazer a leitura do potenciômetro (V2) utilizando o comando `analogRead` do pin A0, anteriormente definindo como `potpin`. O valor obtido com o `analogRead`, encontra-se em bytes (0-1023), por isso faz-se um map deste valor para passá-lo para uma posição que o servo consiga ler e executar $[-90^{\circ}, +90^{\circ}]$, para evitar uma “deathzone” entre a posição 0 a 0.5 é possível fazer um map de $[0^{\circ}, 180^{\circ}]$ (parte do código comentada), para “enviar” a posição ao servo que ele necessita realizar é necessário usar o comando `nameServo.write()`, sendo que o `nameServo` foi o nome atribuído no código ao servo.

No void loop também é feita a *print* da posição do potenciômetro e do servo (na consola-

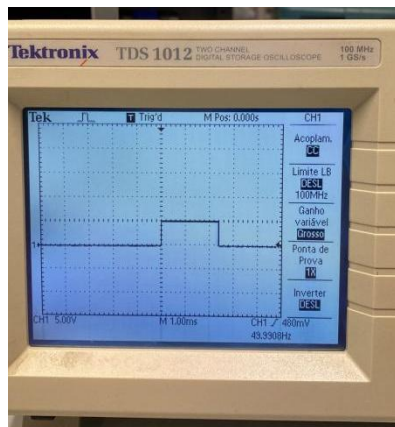
Serial.println()) para assim o utilizador deste código ter uma melhor perceção do bom funcionamento do mesmo.

Prints do *output* do código para a consola(posição 0 e 1 do potenciômetro correspondem á posição -90° e $+90^\circ$ do servo):

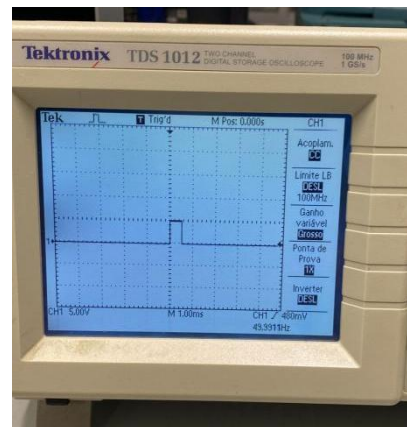
0.00
-90

1.00
90

Osciloscópio:



Servo a $+90^\circ$



Servo a -90°

Circuito 2:

```
const int TRIG = 7;
const int ECHO = 8;
unsigned long width;
unsigned long distance;
void setup()
{
  Serial.begin(9600);
  pinMode(TRIG,OUTPUT);
  pinMode(ECHO,INPUT);
  pinMode(4,OUTPUT);
}
void loop()
{
  sonar();
  piezoC2();
}
void sonar()
{
  digitalWrite(TRIG,HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG,LOW);
  width = pulseIn(ECHO,HIGH);
  distance = microtocenti(width);
  Serial.println(distance);
  delay(100);
}
long microtocenti (long micro)
{
  return micro/29/2; // v do som 340m/s = 29 micros por centi
}
void piezoC2()
{
  if(distance>=5 && distance <=50)
  {
    tone(4, map(distance,5,50, 5, 200)/10);
  } else
  {
    Serial.println("outside of scope");
    noTone(4);
  }
}
```

Neste circuito, criou-se um código para alterar a frequência de som intermitente do buzzer no intervalo de [0.5,20] Hz através da distância medida pelo sensor (sonar), esta distância tinha de estar entre o seguinte intervalo para o buzzer emitir som: [5, 50] cm.

Para atingir este objetivo, inicialmente no void setup indicou-se os diferentes pinMode's, sendo o *pin* TRIG um output para assim iniciar o pulso, o *pin* ECHO um input para assim obter o comprimento do pulso e o *pin* 4 um output para assim controlar o buzzer, também foi necessário criar três funções: void sonar () que inicialmente utiliza o comando digitalWrite (TRIG,HIGH) para ativar o pino do *trigger* e depois de um delay de 10 μ s realiza-se um digitalWrite(TRIG,LOW) para assim desativar o *pin*.

Para obter o comprimento do sinal é necessário o pulseIn (ECHO,HIGH) que vai retornar o tempo que demorou a transição do estado do pulso entre HIGH e o LOW, em microsegundos. De seguida é necessário transformar o valor de uma variável temporal (obtida pelo pulseIn) para uma medida em distância, logo utiliza-se a função long microtoci (microtoci) que recebe um parâmetro (um valor em microsegundos) para assim transformá-lo numa distância utilizando a seguinte formula: $\text{micro}/29/2$. Por fim na função void sonar, chama-se a função microtoci com a variável width como parâmetro para obter o valor da distancia que o sonar está a ler, nesta função também é feito um Serial.println da distância e delay de 100 milisegundos.

A terceira função criada neste programa é a void piezoC2 que tem como objetivo ligar o buzzer e fazer variar a sua frequência através do valor da distância obtida no void sonar, para isso faz-se inicialmente um if, no caso da distância encontrar-se no intervalo desejado [5, 50] cm, o buzzer é ligado através do comando tone() tendo como parâmetros o pin 4 e um map entre a variável distance que varia entre 5 e 50 cm e a frequência que queremos obter que varia entre 0.5

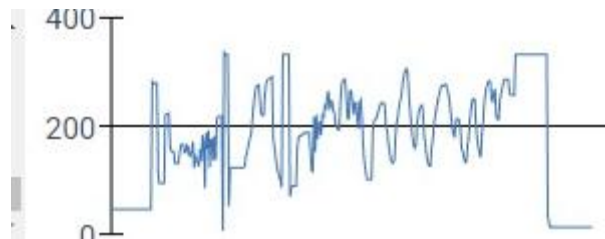
e 20 Hz, devido a este intervalo ser pequeno faz-se uma multiplicação dos valores da frequência por 10 e fora do map divide-se outra vez por 10 para assim obter uma melhor representação da frequência desejada. No caso da distância não encontrar-se no intervalo desejado, faz-se um else para desligar o buzzer usando o noTone do *pin* 4 e faz-se Serial.println (“outside of scope”) para mostrar ao *user*, o porquê do buzzer não emitir som.

Para concluir este código é necessário chamar as funções sonar() e piezoC2() no void loop().

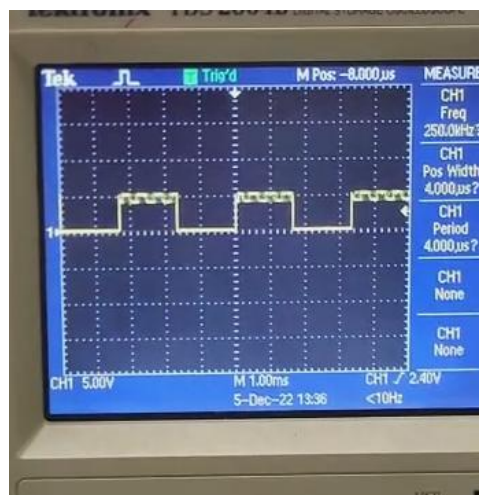
Prints do *output* do código para a consola (valor da distância entre 0 e 325, sendo estes o *range* do sonar):

```
outside of scope
325                                40
```

Plotter (variação da distância recebida pelo sonar ao longo do tempo, tendo como valor máximo o range de 325 do sonar):



Osciloscópio:



Circuito 3:

```
void setup()
{
  Serial.begin(9600);
  pinMode(2,OUTPUT);
}
void loop()
{
  float brilho=ldr();
  Serial.println(brilho);
  delay(1000);
  piezo(brilho);
}
void piezo(float brilho)
{
  tone(2,(map(brilho,0,100,200,10000)));
}
float ldr()
{
  float v=5.00*analogRead(A1)/1023;
  float RLDR=10.0*(5.00/v-1);
  float brilho = pow(RLDR/23.48,-1/0.837);
  return brilho;
}
```

Para controlar a frequência do piezo no intervalo de [0.2,10] kHz através do sensor LDR é necessário inicialmente escrever no void setup o comando pinMode(2,OUTPUT) para assim o arduino conseguir “comunicar” com o piezo.

Neste código é necessário criar duas funções: a float ldr() que transforma o valor analogRead do pin A1 em voltagem, para depois usar esse valor para calcular o resistência do LDR e brilho que este recebe usando as seguintes fórmulas $v=5.00*\text{analogRead}(A1)/1023$; $RLDR = 10.0*(5.00/v-1)$; $\text{brilho} = \sqrt[0.837]{RLDR/23.48}$. Por fim, esta função retorna o brilho do LDR. A segunda função criada é do tipo void e chama-se piezo que recebe como parâmetro um

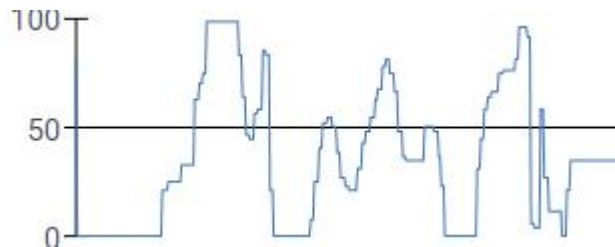
float (o brilho do LDR), esta função executa um tone do pin 2 sendo o valor da frequência um map do brilho com valores de 0 a 100 para a frequência pretendida 200 a 10000 Hz.

No void loop estas funções são chamadas, sendo que o valor da função ldr() é guardada numa variável (float brilho) para assim fazer-se Serial.println desse valor e utilizá-lo na função piezo(brilho).

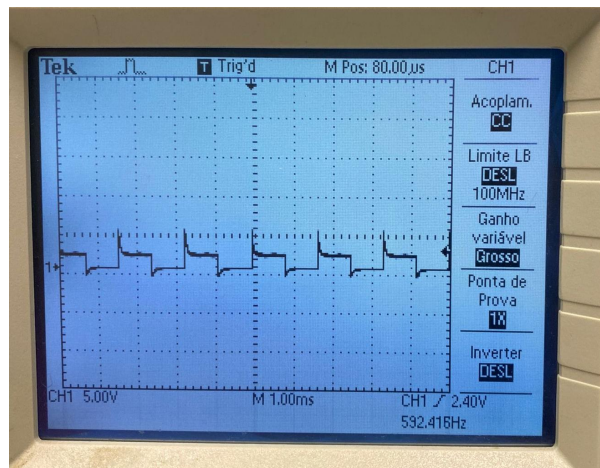
Prints do output do código para a consola (valor do brilho em percentagem):

```
0.09  
98.65
```

Plotter (variação do brilho do LDR em percentagem):



Osciloscópio:



Circuito 4:

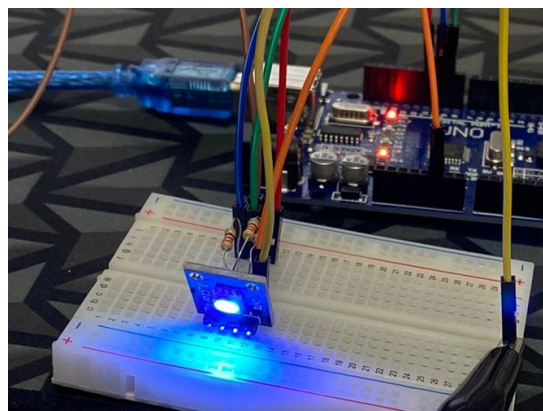
```
void setup()
{
  Serial.begin(9600);
  pinMode(11,OUTPUT);
  pinMode(10,OUTPUT);
  pinMode(9,OUTPUT);
}
void loop()
{
  led_rgb();
}
void led_rgb()
{
  int r=0,g=0,b=0;
  while(Serial.available()==0){}
  r=Serial.parseInt();
  while(Serial.available()==0){}
  g=Serial.parseInt();
  while(Serial.available()==0){}
  b=Serial.parseInt();
  if(r>100 || g>100 || b>100)
  {
    Serial.println("outside of scope");
  } else
  {
    analogWrite(9,map(r,0,100,0,255));
    analogWrite(11,map(g,0,100,0,255));
    analogWrite(10,map(b,0,100,0,255));
    Serial.print("R");
    Serial.print(r);
    Serial.print("G");
    Serial.print(g);
    Serial.print("B");
    Serial.print(b);
    Serial.println("");
  }
}
```

Para controlar a cor do LED RGB através da introdução de valores 0-100% para cada cor (vermelho, verde e azul) e enviar na consola em formato “R30G40B80” é necessário indicar no void setup os pinModes’s sendo que os três *pins* do LED (9,10 e 11) são configurados como outputs para assim enviar informação para o RGB.

De seguida, é criada a função void led_rgb() que utilizando o ciclo while com o Serial.available()==0, como condição, e após o ciclo Serial.parseInt() vamos dar valores para as várias variáveis r, g, b, depois confirmamos se estas variáveis estão no *range* desejado, ou seja, no caso de o valor introduzido na consola for superior a 100 em qualquer uma das três variáveis o LED não faz nada e é feito um Serial.println(“outside of scope”), no caso dos valores introduzidos estarem em percentagem é feito um analogWrite dos respetivos *pins* 9,10 e 11 (9-*red*, 10-*blue* e 11-*green*) e um map da respetiva variável, r (*red*), b (*blue*) e g (*green*) de 0 a 100 para 0 a 255 bytes. Por fim é feito Serial.println() das seguintes strings (“R”, ”G”, ”B”) e os valores das variáveis r, g e b.

Por fim no void loop, chama-se a função led_rgb().

Print do *output* do código para a consola (valor em percentagem de cada cor) e imagem da cor do LED RGB:

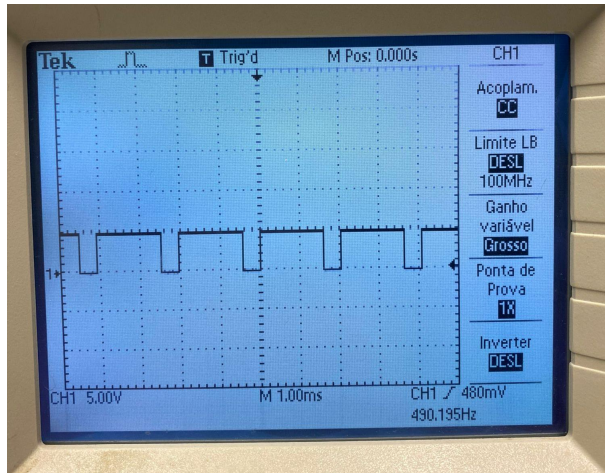


R30G40B80

(No caso de um dos valores introduzidos estar fora do range desejado)

outside of scope

Osciloscópio:



Circuito 5:

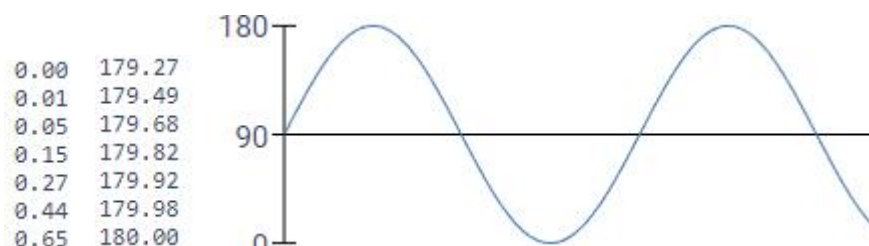
```
#include <Servo.h>
#define servo 9
Servo nameServo;
void setup()
{
    Serial.begin(9600);
    nameServo.attach(9);
}
void loop() {
    unsigned long t=millis();
    nameServo.write(simuladorservo(t)); //varreamento do servo
    Serial.println(simuladorservo(t)); //ver no plotter a onda criada
    delay(15);
}
float simuladorservo(unsigned long t) // onda sinusoidal
{
    const float ydc=90, yac=90;
    const int T=5;
    return ydc+yac*sin(6.28*t/1000/T);
}
```

Neste circuito escolheu-se o atuador servo, para varrer o respetivo parâmetro em forma de uma onda simulada com período de 5 segundos.

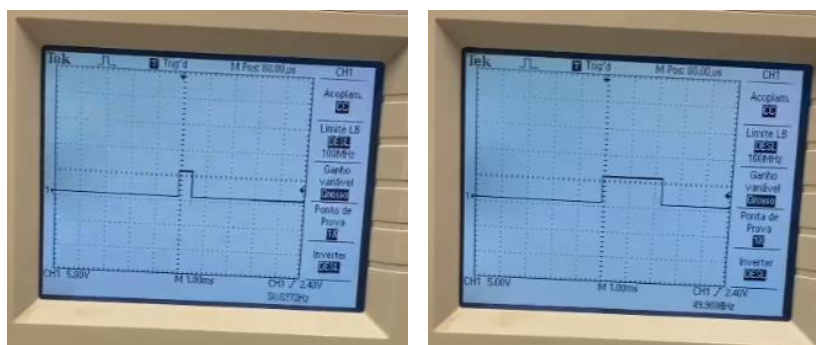
Para atingir este objetivo é necessário introduzir no void setup, utilizando a biblioteca do servo, o comando `nameServo.attach(9)` para ligar o pino 9 a este atuador. De seguida cria-se a função `float simuladorservo()` que recebe como parâmetro o tempo em milissegundos (`unsigned long t`). De seguida cria-se três constantes `ydc`, `yac` e `T`, sendo que `ydc` e `yac` baseiam-se nas seguintes fórmulas: $ydc = \frac{y_{max} + y_{min}}{2} = 90$, $yac = \frac{y_{max} - y_{min}}{2} = 90$ e o $T=5$ segundos, pois esse é o período desejado. Esta função irá retornar a seguinte onda sinusoidal $ydc + yac * \sin(2\pi * t / 1000 / T)$.

No void `loop()` cria-se uma variável `unsigned long t` que utilizando o comando `millis()` guarda os milissegundos registados pelo arduino, depois usou-se o comando `nameServo.write()` sendo o seu parâmetro a função `simuladorservo(t)`, esta execução irá permitir a varreamento do atuador. Por fim faz-se um `Serial.println` da função `simuladorservo(t)` para obter no plotter a função sinusoidal criada.

Prints do output do código para a consola (posição do servo) e Plotter (função sinusoidal):



Osciloscópio (primeira imagem representa o servo no valor mínimo e segunda no seu valor máximo):



Conclusão: A atividade laboratorial cumpriu o seu objetivo, pois os alunos conseguiram realizar o código com os novos atuadores e sensores (servo, buzzer, piezo, LED RGB e sonar) e apresentar um sentido crítico sobre o mesmo.

Durante a atividade laboratorial, o grupo não apresentou grandes dificuldades na execução da atividade, no entanto identificou-se uma anomalia com o LED RGB pois este tinha os canais *red and green* trocados. Sendo que isto teve uma fácil correção, sendo esta a troca dos *pins* no arduino.